Safety in Numbers

JASON TURNER



Cppcon
The C++ Conference

2026

September 12 - 18

Copyright Jason Turner

@lefticus

emptycrate.com/go.html

Safety In Numbers



Jason Turner

C++ Weekly

- Weekly videos since March, 2016
- 125k+ subscribers, 520+ weeks straight

https://www.youtube.com/@cppweekly



Jason Turner

- Author
  - C++ Best Practices, C++23 Best Practices
  - OpCode, Copy and Reference, Object Lifetime Puzzlers
  - Programming Puzzles Big Book
  - **https://amzn.to/3xWh8Ox**
  - **https://leanpub.com/u/jason_turner**



Jason Turner

- Developer
  - https://cppbestpractices.com
  - https://github.com/lefticus
  - https://github.com/cpp-best-practices
- Microsoft MVP for C++, 2015-present



Jason Turner - Training

https://emptycrate.com/training.html

How to get my training:

1. Have me come to your company on-site for dynamic customized training where you already are - generally the most economical option for groups (CA, DE, NL, RO, CZ, JP, US, PL, SE, …)
2. Come to a conference workshop
   - ACCU On Sea (Folkestone, UK, ~June)
   - CppCon (Aurora, CO, US, ~Sept)
   - NDC TechTown (Kongsberg, NO, ~Sept)
   - C++ Under the Sea (Breda, NL, ~Oct)
   - And possibly others



My Motivations



My Motivations

- The world runs on software. Basically everything does.



My Motivations

- The world runs on software. Basically everything does.
- Have you ever had to reboot your toaster? (I have!)



My Motivations

- The world runs on software. Basically everything does.
- Have you ever had to reboot your toaster? (I have!)
- I want to live in a world with better, safer software, cars, spaceships, toasters…



My Motivations

- The world runs on software. Basically everything does.
- Have you ever had to reboot your toaster? (I have!)
- I want to live in a world with better, safer software, cars, spaceships, toasters…
- This is why I've given away so much material and continue to do so.



My Motivations

- The world runs on software. Basically everything does.
- Have you ever had to reboot your toaster? (I have!)
- I want to live in a world with better, safer software, cars, spaceships, toasters…
- This is why I've given away so much material and continue to do so.
- This is also why I focus on Best Practices in my training material.



My Motivations

- The world runs on software. Basically everything does.
- Have you ever had to reboot your toaster? (I have!)
- I want to live in a world with better, safer software, cars, spaceships, toasters…
- This is why I've given away so much material and continue to do so.
- This is also why I focus on Best Practices in my training material.
- The work you are doing matters. It affects peoples lives!



Jason Turner

https://emptycrate.com/go.html



About my Talks

- Avoid sitting in the back
- Please interrupt and ask questions, yell things out, I'll repeat it for the room
- This is approximately how my training days look, as interactive as reasonable



Safety In Numbers



Safety in Numbers



Safety in Numbers

- C and C++ today have many tools that help you write safe numeric code



Safety in Numbers

- C and C++ today have many tools that help you write safe numeric code
- The problem is that they are spread across the two different standards!



Safety in Numbers

- C and C++ today have many tools that help you write safe numeric code
- The problem is that they are spread across the two different standards!
- In this talk we're going to do a survey of these tools and see how we might fit them together



Safety in Numbers

- C and C++ today have many tools that help you write safe numeric code
- The problem is that they are spread across the two different standards!
- In this talk we're going to do a survey of these tools and see how we might fit them together
- This is not going to present a completed solution for you to use



Safety in Numbers

- C and C++ today have many tools that help you write safe numeric code
- The problem is that they are spread across the two different standards!
- In this talk we're going to do a survey of these tools and see how we might fit them together
- This is not going to present a completed solution for you to use
- If you want a real-world type-safe tool for working with numbers, look at **https://github.com/mpusz/mp-units**





Why do we care?



Quick Review

```cpp
#include <cstdint>

int main()
{
  const std::uint8_t x = 175;
  const std::uint8_t y = 125;
  const auto result = x + y;
  return result; /// what is returned?
}
```

https://godbolt.org/z/E3Y96GKhP



Quick Review

```cpp
#include <cstdint>

int main()
{
  const std::uint8_t x = 175;
  const std::uint8_t y = 125;
  const auto result = x + y;
  return result; /// what is returned?
}
```

https://godbolt.org/z/E3Y96GKhP

It's an `int` with value `300` of course!



Quick Review

```
1  #include <cstdint>
2
3  int main()
4  {
5    const unsigned int x = 0;
6    const int y = -1;
7
8    return y < x; /// what is returned?
9  }
```

https://godbolt.org/z/cvKorTEP5



Quick Review

```
1  #include <cstdint>
2
3  int main()
4  {
5    const unsigned int x = 0;
6    const int y = -1;
7
8    return y < x; /// what is returned?
9  }
```

https://godbolt.org/z/cvKorTEP5

False of course!



Quick Review

```cpp
#include <cstdint>

int main()
{
  const std::uint8_t x = 0;
  const std::uint8_t y = 1;

  const std::uint8_t result = x - y;

  return result; /// what is returned?
}
```

https://godbolt.org/z/Y5cv7qTfh



Quick Review

```
1  #include <cstdint>
2
3  int main()
4  {
5    const std::uint8_t x = 0;
6    const std::uint8_t y = 1;
7
8    const std::uint8_t result = x - y;
9
10   return result; /// what is returned?
11 }
```

https://godbolt.org/z/Y5cv7qTfh

255!



Quick Review

```cpp
#include <cstdint>

int main()
{
  const std::uint8_t x = 0;
  const std::uint8_t y = 1;

  const std::uint8_t result = x - y;

  return result; /// what is returned?
}
```

https://godbolt.org/z/Y5cv7qTfh

255!

This is well defined behavior.



Quick Review

```
1  #include <cstdint>
2
3  int main()
4  {
5    const std::uint8_t x = 0;
6    const std::uint8_t y = 1;
7
8    const std::uint8_t result = (x - y) >> 1;
9
10   return result; /// what is returned?
11 }
```

https://godbolt.org/z/jh61Yd369



Quick Review

```cpp
#include <cstdint>

int main()
{
  const std::uint8_t x = 0;
  const std::uint8_t y = 1;

  const std::uint8_t result = (x - y) >> 1;

  return result; /// what is returned?
}
```

https://godbolt.org/z/jh61Yd369

255 of course!



Quick Review

```cpp
#include <cstdint>

int main()
{
  const std::uint8_t x = 0;
  const std::uint8_t y = 1;

  const std::uint8_t result = (x - y) >> 1;

  return result; /// what is returned?
}
```

https://godbolt.org/z/jh61Yd369

255 of course!

Implicit promotion of `(x - y)` to an `int`, then shift of an `int >> 1`, then explicit truncation of those bits back into a `uint8_t`!



Quick Review

```cpp
#include <cstdint>

int main()
{
  const auto result = static_cast<std::uint8_t>(256);

  return result; /// what is returned?
}
```

https://godbolt.org/z/3WrGnaWGM



Quick Review

```
1  #include <cstdint>
2
3  int main()
4  {
5    const auto result = static_cast<std::uint8_t>(256);
6
7    return result; /// what is returned?
8  }
```

https://godbolt.org/z/3WrGnaWGM

0 of course!



Quick Review

```cpp
#include <cstdint>
#include <print>

int main()
{
  const int x = 2'100'000'000;

  std::println("{}", x * 2); /// what is printed?
}
```

https://godbolt.org/z/r7dvY7ahd



Quick Review

```cpp
#include <cstdint>
#include <print>

int main()
{
  const int x = 2'100'000'000;

  std::println("{}", x * 2); /// what is printed?
}
```

https://godbolt.org/z/r7dvY7ahd

-94,967,296 of course!





Who says this is the behavior they want?



Who works in a field that needs correct math?



Who currently handles overflow, underflow, etc?



What tools do you use to do this?



Safety in Numbers



Safety in Numbers

- safe comparisons `cmp_`* (C++20) (`constexpr`)



Safety in Numbers

- safe comparisons `cmp_`* (C++20) (`constexpr`)
- `std::in_range` (C++20) (`constexpr`)



Safety in Numbers

- safe comparisons `cmp_`* (C++20) (`constexpr`)
- `std::in_range` (C++20) (`constexpr`)
- saturating math `saturating_`* (C++26) (`constexpr`)



Safety in Numbers

- safe comparisons `cmp_`* (C++20) (`constexpr`)
- `std::in_range` (C++20) (`constexpr`)
- saturating math `saturating_`* (C++26) (`constexpr`)
- checked math `ckd_`* (C23 / C++26)



Safety in Numbers

- safe comparisons `cmp_`* (C++20) (`constexpr`)
- `std::in_range` (C++20) (`constexpr`)
- saturating math `saturating_`* (C++26) (`constexpr`)
- checked math `ckd_`* (C23 / C++26)
- contracts (C++26)



Safety in Numbers

- safe comparisons `cmp_`* (C++20) (`constexpr`)
- `std::in_range` (C++20) (`constexpr`)
- saturating math `saturating_`* (C++26) (`constexpr`)
- checked math `ckd_`* (C23 / C++26)
- contracts (C++26)
- UBSan Minimal Runtime





[utility.intcmp]



[utility.intcmp]

C++20, constexpr. In `<utility>`

- `cmp_equal`
- `cmp_less`
- `cmp_greater`
- `cmp_less_equal`
- `cmp_greater_equal`
- `in_range`



[utility.intcmp]: `cmp_*` functions

These do The Right Thing when comparing integers of different sizes and signs.

```
1  #include <utility>
2
3  int main()
4  {
5    const unsigned int x = 0;
6    const int y = -1;
7
8    return std::cmp_less(y, x); /// true!
9  }
```

https://godbolt.org/z/sPb6K4xs9



[utility.intcmp]: `cmp_*` functions

```
template<class T, class U>
  constexpr bool cmp_equal(T t, U u) noexcept;
```

Mandates: Each of T and U is a signed or unsigned integer type ([basic.fundamental]).

Effects: Equivalent to:

```
using UT = make_unsigned_t<T>;

using UU = make_unsigned_t<U>;
if constexpr (is_signed_v<T> == is_signed_v<U>)
  return t == u;
else if constexpr (is_signed_v<T>)
  return t < 0 ? false : UT(t) == u;
else
  return u < 0 ? false : t == UU(u);
```

https://godbolt.org/z/MfKEoE7EG



A minimally safe wrapper that allows for safe mixed-type comparisons. This doesn't provide "type safety" as we would normally think of it, but it does eliminate one class of errors.

```cpp
#include <compare>
#include <utility>

template<typename ValueType>
struct SafeCompare {
  using value_type = ValueType;

  value_type value;

  template<typename RHS>
  [[nodiscard]] constexpr auto operator<=>(const SafeCompare<RHS> &rhs) const noexcept
      -> std::strong_ordering {
    if (std::cmp_less(value, rhs.value)) {
      return std::strong_ordering::less;
    } else if (std::cmp_equal(value, rhs.value)) {
      return std::strong_ordering::equal;
    } else {
      return std::strong_ordering::greater;
    }
  }
};
```

https://godbolt.org/z/GaM3sK4nx

What's the generated code like?

```cpp
#include <compare>
#include <utility>

template<typename ValueType> struct SafeCompare {
  using value_type = ValueType;
  value_type value;
  template<typename RHS>
  [[nodiscard]] constexpr auto operator<=>(const SafeCompare<RHS> &rhs) const noexcept
      -> std::strong_ordering {
    // we only need 2, but which and in which order?
    // and which is more likely?
    if (std::cmp_less(value, rhs.value)) {
      return std::strong_ordering::less;
    } else if (std::cmp_equal(value, rhs.value)) {
      return std::strong_ordering::equal;
    } else {
      return std::strong_ordering::greater;
    }
  }
};

auto compare(SafeCompare<int> lhs, SafeCompare<unsigned int> rhs) {
  return lhs <=> rhs;
}
```

https://godbolt.org/z/1da65xn85



[utility.intcmp]: `std::in_range`

```cpp
#include <cstdint>
#include <utility>

int main()
{
  return std::in_range<std::uint8_t>(256); /// false
}
```

https://godbolt.org/z/c391z5xnq



[utility.intcmp]

> *[Note 1: These function templates cannot be used to compare* `byte`, `char`, `char8_t`, `char16_t`, `char32_t`, `wchar_t`, *and* `bool`. *— end note]*

```cpp
#include <cstdint>
#include <utility>

int main()
{
  // works!
  return
    std::in_range<std::uint8_t>(256);
}
```

https://godbolt.org/z/hvM8fvc7h

```cpp
#include <cstdint>
#include <utility>

int main()
{
  // compile error!
  return
    std::in_range<char8_t>(256);
}
```

https://godbolt.org/z/j8sf7EKzz



What's the generated code for `in_range` like?

```cpp
#include <cstdint>
#include <utility>

bool test_in_range(int input)
{
  return std::in_range<std::int8_t>(input);
}
```

https://godbolt.org/z/8r67KG9Pq



[utility.intcmp] - review

C++20, constexpr. In `<utility>`

- `cmp_equal`
- `cmp_less`
- `cmp_greater`
- `cmp_less_equal`
- `cmp_greater_equal`
- `in_range`





[numeric.sat]



[numeric.sat]

`constexpr`, C++26, in `<numeric>`

- `saturating_add`
- `saturating_mul`
- `saturating_div`
- `saturating_sub`
- `saturating_cast`

If the operation were to overflow or underflow, then the max or min result possible is returned instead



[numeric.sat]: `saturating_*`

`saturating_add`, `saturating_mul`, `saturating_div`, `saturating_sub`

(previously called `add_sat`, `mul_sat`, `div_sat`, `sub_sat` before finalization, and we have to use that for our examples today)

Note that these are more strongly typed than most: the inputs must match and the return type matches the inputs.

```cpp
#include <cstdint>
#include <numeric>

int main() {
  const std::uint8_t x = 0;
  const std::uint8_t y = 1;

  // returns 0
  /// (saturating_sub)
  return std::sub_sat(x, y);
}
```

https://godbolt.org/z/9nM87rGqh

```cpp
#include <cstdint>
#include <numeric>

int main() {
  const std::int8_t x = 127;
  const std::int8_t y = 4;

  // returns 127
  /// (saturating_add)
  return std::add_sat(x, y);
}
```

https://godbolt.org/z/Tzd93szh5



[numeric.sat]: `saturating_*`

What's the generated code like?

```
1  #include <cstdint>
2  #include <numeric>
3
4  auto saturating_add(std::uint64_t x, std::uint64_t y) {
5    /// (saturating_add)
6    return std::add_sat(x, y);
7  }
```

https://godbolt.org/z/GojnTT1Ed



[numeric.sat]: `saturating_cast<>`

```cpp
#include <cstdint>
#include <numeric>

int main() {
  // returns 255
  return std::saturate_cast<std::uint8_t>(256);
}
```

https://godbolt.org/z/Wv3Pr7zoM



[numeric.sat]: `saturating_cast<>`

What's the generated code like?

```cpp
#include <cstdint>
#include <numeric>

auto sat_cast(const int input) {
  return std::saturate_cast<std::int8_t>(input);
}
```

https://godbolt.org/z/xcE4Tv7xz

Note: I intentionally chose one of the more expensive integer range checks for this example.



[numeric.sat]: An example wrapper type

```cpp
#include <numeric>

template<typename ValueType>
struct Saturating {
  using value_type = ValueType;
  value_type value;

  [[nodiscard]] constexpr auto operator+(const ValueType rhs) noexcept {
    /// (saturating_add)
    return Saturating{std::add_sat(value, rhs.value)};
  }

  constexpr auto &operator+=(const ValueType rhs) noexcept {
    return *this = *this + rhs;
  }

  // operators +, -, *, / are all easily implementable
  // but how do we consistently implement %, <<, >>, or does it matter?
};
```

https://godbolt.org/z/jM416Mo1d



[numeric.sat] - review

`constexpr`, C++26, in `<numeric>`

- `saturating_add`
- `saturating_mul`
- `saturating_div`
- `saturating_sub`
- `saturating_cast`

If the operation were to overflow or underflow, then the max or min result possible is returned instead





[numerics.c.ckdint]



[numerics.c.ckdint]

C++26, actually imported from C, not `constexpr` (and not properly in `std::`?). In `<stdckdint.h>`

- `ckd_add`
- `ckd_sub`
- `ckd_mul`



[numerics.c.ckdint]

C++26, actually imported from C, not `constexpr` (and not properly in `std::`?). In `<stdckdint.h>`

- `ckd_add`
- `ckd_sub`
- `ckd_mul`

The types do not have to match, which I believe is intentional.



[numerics.c.ckdint]

C++26, actually imported from C, not `constexpr` (and not properly in `std::`?). In `<stdckdint.h>`

- `ckd_add`
- `ckd_sub`
- `ckd_mul`

The types do not have to match, which I believe is intentional.

The interface seems pretty limited, and wrapped from C without much other C++ considerations except for making them templates



[numerics.c.ckdint]



[numerics.c.ckdint]

- Only these 3 operations



[numerics.c.ckdint]

- Only these 3 operations
- Not `constexpr`



[numerics.c.ckdint]

- Only these 3 operations
- Not `constexpr`
- Not properly namespaced?



[numerics.c.ckdint]

- Only these 3 operations
- Not `constexpr`
- Not properly namespaced?

However



[numerics.c.ckdint]

- Only these 3 operations
- Not `constexpr`
- Not properly namespaced?

However

- It's very difficult to do this detection portably, so this is a nice tool to have



[numerics.c.ckdint]

- Only these 3 operations
- Not `constexpr`
- Not properly namespaced?

However

- It's very difficult to do this detection portably, so this is a nice tool to have

The math is done as if it has infinite precision so it knows if the result != the expected result.



[numerics.c.ckdint]: `ckd_mul`

```cpp
#include <print>
#include <stdckdint.h>

int main()
{
  const int x = 2'100'000'000;
  const int y = 2;

  int result = 0;

  // overflow or underflow or any misrepresentation
  // (it's kind of in negative land)
  bool does_not_correctly_represent_result = ckd_mul(&result, x, y);

  std::print("({} * {}) == {} (Overflow: {})",
             x, y, result, does_not_correctly_represent_result);
}
```

https://godbolt.org/z/4aqjexMf9



[numerics.c.ckdint]: `ckd_mul`

```cpp
#include <print>
#include <stdckdint.h>

int main()
{
  const int x = 2'100'000'000;
  const int y = 2;

  int result = 0;

  // overflow or underflow or any misrepresentation
  // (it's kind of in negative land)
  bool does_not_correctly_represent_result = ckd_mul(&result, x, y);

  std::print("({} * {}) == {} (Overflow: {})",
             x, y, result, does_not_correctly_represent_result);
}
```

https://godbolt.org/z/4aqjexMf9

```
(2100000000 * 2) == -94967296 (Overflow: true)
```



[numerics.c.ckdint]: `ckd_div`?

There is no `ckd_div`, but it might seem logical for there to be one.



[numerics.c.ckdint]: `ckd_div`?

There is no `ckd_div`, but it might seem logical for there to be one.

```
1  #include <concepts>
2  #include <stdckdint.h>
3
4  template<std::integral Integral>
5  auto ckd_div(Integral lhs, Integral rhs) {
6    const auto result = lhs / rhs;
7    if (rhs * result == lhs) {
8      return result;
9    } else {
10     //...?
11   }
12 }
```

https://godbolt.org/z/hMjsP5Ed9



[numerics.c.ckdint]: `ckd_div`?

There is no `ckd_div`, but it might seem logical for there to be one.

```
1  #include <concepts>
2  #include <stdckdint.h>
3
4  template<std::integral Integral>
5  auto ckd_div(Integral lhs, Integral rhs) {
6    const auto result = lhs / rhs;
7    if (rhs * result == lhs) {
8      return result;
9    } else {
10     //...?
11   }
12 }
```

https://godbolt.org/z/hMjsP5Ed9

However, this does not have the same semantics as the other `ckd_*` functions in that it does not check for overflow or underflow, it checks for whether or not there was any remainder.



[numerics.c.ckdint]: `ckd_div`?

```cpp
#include <concepts>
#include <stdckdint.h>

template<std::integral Integral>
auto ckd_div(Integral lhs, Integral rhs) {
  const auto result = static_cast<Integral>(lhs / rhs);

  Integral check{};

  // overflow or underflow or any misrepresentation
  bool does_not_correctly_represent_result = ckd_mul(&check, result, rhs);

  if (not does_not_correctly_represent_result) {
    return result;
  } else {
    //...?
  }
}
```

https://godbolt.org/z/Pzd8vff4T

What do we do in the “error” case in this case, or any of these cases?



One Possible Idea?



```cpp
#include <concepts>
#include <stdckdint.h>
#include <variant>

template<std::integral Integral> struct Rational {
  Integral numerator;
  Integral denominator;
};

template<std::integral Integral>
std::variant<Integral, Rational<Integral>> ckd_div(Integral lhs, Integral rhs) {
  /// integral division
  const auto result = static_cast<Integral>(lhs / rhs);
  Integral check{};

  /// checked multiplication
  bool does_not_correctly_represent_result = ckd_mul(&check, result, rhs);

  if (not does_not_correctly_represent_result && check == lhs) {
    /// if there was no remainder and no overflow, then it's the
    /// correct integral result
    return result;
  } else {
    /// else return a rational number that loses no data
    /// or precision
    Rational<Integral>{lhs, rhs};
  }
} /// thoughts on this?
```

https://godbolt.org/z/fKbc3vx6E



What Do You Do?



What Do You Do?

- How do you weigh the runtime / compile time costs?



What Do You Do?

- How do you weigh the runtime / compile time costs?
- Do you want to stay within your specified types?



What Do You Do?

- How do you weigh the runtime / compile time costs?
- Do you want to stay within your specified types?
- How strong of typing do you want?



What Do You Do?

- How do you weigh the runtime / compile time costs?
- Do you want to stay within your specified types?
- How strong of typing do you want?
- How do you handle the error cases?



Contracts

```
1  #include <stdckdint.h>
2
3  // overflow or underflow or any misrepresentation
4  template<std::integral ValueType>
5  bool adds_correctly(const ValueType lhs, const ValueType rhs) {
6    ValueType result{};
7    return !ckd_add(&result, lhs, rhs);
8  }
9
10 template<std::integral ValueType>
11 struct Checked_Math {
12   ValueType value;
13
14   // cannot be constexpr?
15   Checked_Math operator+(const Checked_Math rhs) const
16     pre(adds_correctly(value, rhs.value))
17   {
18     return Checked_Math{value + rhs.value};
19   }
20 };
```

https://godbolt.org/z/Kq61oWz8T

How do we make this `constexpr` friendly?



Contract Checker

```cpp
#include <stdckdint.h>

// overflow or underflow or any misrepresentation
template<std::integral Result, std::integral LHS, std::integral RHS>
[[nodiscard]] constexpr bool adds_correctly(const LHS lhs,
                                            const RHS rhs) {
  Result result{};
  if consteval {
    if constexpr (std::is_signed_v<std::common_type_t<LHS, RHS>>) {
      result = lhs + rhs; // if there's signed integer overflow
                          // consteval will catch it and fail compile
      return true;
    } else {
      result = lhs + rhs;
      return result > lhs && result > rhs;
    }
  } else {
    return !ckd_add(&result, lhs, rhs);
  }
}
```

https://godbolt.org/z/YP7qq75K3

I think this works correctly for `consteval` and non-`consteval` cases



[numerics.c.ckdint] - Review

C++26, actually imported from C, not `constexpr` (and not properly in `std::`?). In `<stdckdint.h>`

- `ckd_add`
- `ckd_sub`
- `ckd_mul`

The types do not have to match, which I believe is intentional.

The interface seems pretty limited, and wrapped from C without much other C++ considerations except for making them templates



One Type to Rule Them All?



Maybe not?



Where to go from here?



Where to go from here?



Where to go from here?

- Do we try to put `ckd_*`, `saturate_*`, and `cmp_*` all in one place and time?



Where to go from here?

- Do we try to put `ckd_*`, `saturate_*`, and `cmp_*` all in one place and time?
- I tried merging them into one class that let you choose what you wanted…



Where to go from here?

- Do we try to put `ckd_*`, `saturate_*`, and `cmp_*` all in one place and time?
- I tried merging them into one class that let you choose what you wanted…
- It ended up feeling like a *Bad Idea*.



Where to go from here?

- Do we try to put `ckd_*`, `saturate_*`, and `cmp_*` all in one place and time?
- I tried merging them into one class that let you choose what you wanted…
- It ended up feeling like a *Bad Idea*.
- Better might be composable versions of these?



Where to go from here?

- Do we try to put `ckd_*`, `saturate_*`, and `cmp_*` all in one place and time?
- I tried merging them into one class that let you choose what you wanted…
- It ended up feeling like a *Bad Idea*.
- Better might be composable versions of these?
- How / when do you use UBSan minimal runtime overflow checks on top of this?



Where to go from here?

The more I think about this the more I think these ideas are mutually exclusive.



Where to go from here?

The more I think about this the more I think these ideas are mutually exclusive.

- Saturating math cannot overflow



Where to go from here?

The more I think about this the more I think these ideas are mutually exclusive.

- Saturating math cannot overflow
- Safe comparisons break some levels of type safety?



Where to go from here?

The more I think about this the more I think these ideas are mutually exclusive.

- Saturating math cannot overflow
- Safe comparisons break some levels of type safety?
- If you saturate - the overflow case is “obvious” (it saturates)



Where to go from here?

The more I think about this the more I think these ideas are mutually exclusive.

- Saturating math cannot overflow
- Safe comparisons break some levels of type safety?
- If you saturate - the overflow case is “obvious” (it saturates)
- But if you check for overflow, you have to decide how to handle the error cases





UBSan “Minimal Runtime”



UBSan “Minimal Runtime”

> *There is a minimal UBSan runtime available suitable for use in production environments. This runtime has a small attack surface. It only provides very basic issue logging and deduplication, and does not support -fsanitize=vptr checking.*

This can be used for checking:

- enum out of range
- signed/unsigned conversions and promotions
- signed/unsigned integer overflow
- non-boolean bools
- 

UBSan “Minimal Runtime”

UBSan Pros:

- No code changes
- Can choose between logging / errors, etc

UBSan Cons:

- No visibility in the code that you’re relying on it
- No possibility to handle inside of the C++ code when an error happens
- Not available on all platforms



Does anyone ship with UBSan today?



Only ship “minimal runtime” version if you do.



Does anyone ship with ASAN today?



💣 Don’t do that!





Contracts



Handling Over/Underflow

`pre` condition

```
1  //nolintfile
2  template<typename ValueType>
3  struct Ckd
4  {
5    ValueType value;
6
7    [[nodiscard]] constexpr Ckd operator+(const Ckd rhs) const
8      pre(can_add_safely<ValueType>(value, rhs.value))
9    {
10     return Ckd{value + rhs.value};
11   }
12 };
```

https://godbolt.org/z/bMjG6TzEz



Handling Over/Underflow

`pre` condition

```
1  //nolintfile
2  template<typename ValueType>
3  struct Ckd
4  {
5    ValueType value;
6
7    [[nodiscard]] constexpr Ckd operator+(const Ckd rhs) const
8      pre(can_add_safely<ValueType>(value, rhs.value))
9    {
10     return Ckd{value + rhs.value};
11   }
12 };
```

https://godbolt.org/z/bMjG6TzEz

This could work, I think, but it kind of duplicates work.



Handling Over/Underflow

`post` condition

```
1  template<typename ValueType>
2  struct Ckd
3  {
4    ValueType value;
5
6    [[nodiscard]] constexpr Ckd operator+(const Ckd rhs) const
7      post(result: std::cmp_equal(result.value, promoted_add(rhs.value, value)))
8    {
9      return Ckd{value + rhs.value};
10   }
11 };
```

https://godbolt.org/z/rodY5bYG1



Handling Over/Underflow

`post` condition

```
1  template<typename ValueType>
2  struct Ckd
3  {
4    ValueType value;
5
6    [[nodiscard]] constexpr Ckd operator+(const Ckd rhs) const
7      post(result: std::cmp_equal(result.value, promoted_add(rhs.value, value)))
8    {
9      return Ckd{value + rhs.value};
10   }
11 };
```

https://godbolt.org/z/rodY5bYG1

Kind of works, up until I'm trying to add 64bit ints that might overflow.



Handling Over/Underflow

`contract_assert`

```
1  template<typename ValueType>
2  struct Ckd
3  {
4    ValueType value;
5
6    [[nodiscard]] constexpr Ckd operator+(const Ckd rhs) const
7    {
8      // intentionally not allowing promotion probably?
9      ValueType result{};
10
11     contract_assert(!std::ckd_add(&result, value, rhs.value));
12
13     return Ckd{result};
14   }
15 };
```

https://godbolt.org/z/3rqx8ef5P



Handling Over/Underflow

`contract_assert`

```
1  template<typename ValueType>
2  struct Ckd
3  {
4    ValueType value;
5
6    [[nodiscard]] constexpr Ckd operator+(const Ckd rhs) const
7    {
8      // intentionally not allowing promotion probably?
9      ValueType result{};
10
11     contract_assert(!std::ckd_add(&result, value, rhs.value));
12
13     return Ckd{result};
14   }
15 };
```

https://godbolt.org/z/3rqx8ef5P

How do we like this?



```cpp
#include <numeric>
#include <utility>
// safe, but still relatively weak
// SafeNumber = SN for tersity
// pre-standard names used for sat
template<std::integral ValueType>
struct SN {
  ValueType value;
  // constexpr, nodiscard, noexcept, etc
  SN operator+(SN rhs) { return SN{std::add_sat(value, rhs.value)}; }
  SN operator-(SN rhs) { return SN{std::sub_sat(value, rhs.value)}; }
  SN operator*(SN rhs) { return SN{std::mul_sat(value, rhs.value)}; }
  SN operator/(SN rhs) { return SN{std::div_sat(value, rhs.value)}; }

  // plus others, safe shifts, etc.
  template<typename RHS>
  [[nodiscard]] constexpr auto operator<=>(const SN<RHS> &rhs) const noexcept
      -> std::strong_ordering {
    if (std::cmp_less(value, rhs.value)) {
      return std::strong_ordering::less;
    } else if (std::cmp_greater(value, rhs.value)) {
      return std::strong_ordering::greater;
    } else {
      return std::strong_ordering::equal;
    }
  }
};
```

https://godbolt.org/z/EP6K6q1eh



Final overview



Final overview

Choosing behavior:

- `std::cmp_equal`
- `std::cmp_less`
- `std::cmp_greater`
- `std::cmp_less_equal`
- `std::cmp_greater_equal`
- `std::in_range`



Final overview

Choosing behavior:

- `saturating_add`
- `saturating_mul`
- `saturating_div`
- `saturating_sub`
- `saturating_cast`



Final overview

Enforcing correctness:

- `ckd_add`
- `ckd_sub`
- `ckd_mul`
- contract enforcement
- UBSan minimal runtime enforcement



Takeaways



Takeaways

- Understand your problem domain



Takeaways

- Understand your problem domain
- Understand your risks



Takeaways

- Understand your problem domain
- Understand your risks
- Understand where computations can (and will) go wrong



Takeaways

- Understand your problem domain
- Understand your risks
- Understand where computations can (and will) go wrong
- Build these stronger typed things and make sure you use them!





Safety In Numbers



Jason Turner

C++ Weekly

- Weekly videos since March, 2016
- 125k+ subscribers, 520+ weeks straight

https://www.youtube.com/@cppweekly



Jason Turner

- Author
  - C++ Best Practices, C++23 Best Practices
  - OpCode, Copy and Reference, Object Lifetime Puzzlers
  - Programming Puzzles Big Book
  - **https://amzn.to/3xWh8Ox**
  - **https://leanpub.com/u/jason_turner**



Jason Turner

- Developer
  - https://cppbestpractices.com
  - https://github.com/lefticus
  - https://github.com/cpp-best-practices
- Microsoft MVP for C++, 2015-present



Jason Turner - Training

https://emptycrate.com/training.html

How to get my training:

1. Have me come to your company on-site for dynamic customized training where you already are - generally the most economical option for groups (CA, DE, NL, RO, CZ, JP, US, PL, SE, …)
2. Come to a conference workshop
   - ACCU On Sea (Folkestone, UK, ~June)
   - CppCon (Aurora, CO, US, ~Sept)
   - NDC TechTown (Kongsberg, NO, ~Sept)
   - C++ Under the Sea (Breda, NL, ~Oct)
   - And possibly others



My Motivations



My Motivations

- The world runs on software. Basically everything does.



My Motivations

- The world runs on software. Basically everything does.
- Have you ever had to reboot your toaster? (I have!)



My Motivations

- The world runs on software. Basically everything does.
- Have you ever had to reboot your toaster? (I have!)
- I want to live in a world with better, safer software, cars, spaceships, toasters…



My Motivations

- The world runs on software. Basically everything does.
- Have you ever had to reboot your toaster? (I have!)
- I want to live in a world with better, safer software, cars, spaceships, toasters…
- This is why I've given away so much material and continue to do so.



My Motivations

- The world runs on software. Basically everything does.
- Have you ever had to reboot your toaster? (I have!)
- I want to live in a world with better, safer software, cars, spaceships, toasters…
- This is why I've given away so much material and continue to do so.
- This is also why I focus on Best Practices in my training material.



My Motivations

- The world runs on software. Basically everything does.
- Have you ever had to reboot your toaster? (I have!)
- I want to live in a world with better, safer software, cars, spaceships, toasters…
- This is why I've given away so much material and continue to do so.
- This is also why I focus on Best Practices in my training material.
- The work you are doing matters. It affects peoples lives!



Jason Turner

https://emptycrate.com/go.html

